Imam Abdulrahman Bin Faisal University
College of Computer Science and Information Technology

ARTI 404 — Natural Language Processing

Lab 7 — Large Language Models (LLMs)

Student Name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1

Lab 7 — Large Language Models (LLMs)

The Three Main Transformer Model Families

Family	Example Models	Main Use
Encoder-only	BERT, RoBERTa	Classification, sentiment analysis, NER
Encoder–decoder	T5, BART	Translation, summarization, question answering
Decoder-only	GPT family	Text generation, conversation, completion

1. Setup

```
# Install the libraries used in this lab (safe to re-run)
import sys
!{sys.executable} -m pip install -q transformers sentencepiece torch

print("Installation complete.")
```

```
Installation complete.
```

```
from transformers import pipeline, set_seed

print("Libraries loaded successfully!")
```

```
Libraries loaded successfully!
```

Part A — Decoder-Only Model

Decoder-only example: GPT-style Text Generation

A decoder-only model predicts the next token based on previous tokens.

We will use **DistilGPT-2**, a small GPT-style model.

```python
generator = pipeline("text-generation", model="distilgpt2")

prompt = "Natural language processing is"

output = generator(prompt, max_new_tokens=30, do_sample=False)

print(output[0]["generated_text"])
```

```
config.json:   0%|          | 0.00/762 [00:00<?, ?B/s]

model.safetensors: reconstructing file:   0%|          |  0.00B /  353MB

model.safetensors: downloading bytes:           |  0.00B

Loading weights:   0%|          | 0/76 [00:00<?, ?it/s]

generation_config.json:   0%|          | 0.00/124 [00:00<?, ?B/s]

tokenizer_config.json:   0%|          | 0.00/26.0 [00:00<?, ?B/s]

vocab.json:   0%|          | 0.00/1.04M [00:00<?, ?B/s]

merges.txt:   0%|          | 0.00/456k [00:00<?, ?B/s]

tokenizer.json:   0%|          | 0.00/1.36M [00:00<?, ?B/s]

Natural language processing is a common feature of the language processing process.
```

Part B — Encoder–Decoder Model

Encoder–decoder example: Instruction / Text-to-Text Generation

Encoder–decoder models contain two main parts:

1. The **encoder** reads the input.
2. The **decoder** produces the output.

We will use **FLAN-T5-small**.

```python
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

prompt = "Translate to French: I love natural language processing."

# Convert text into tokens
output = model.generate(**tokenizer(prompt, return_tensors="pt"))

# Convert tokens back to text
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

```
config.json:   0%|          | 0.00/1.40k [00:00<?, ?B/s]
```

```
tokenizer_config.json:   0%|          | 0.00/2.54k [00:00<?, ?B/s]
```

```
spiece.model: reconstructing file:   0%|          |  0.00B /  792kB
```

```
spiece.model: downloading bytes:           |  0.00B
```

```
tokenizer.json:   0%|          | 0.00/2.42M [00:00<?, ?B/s]
```

```
special_tokens_map.json:   0%|          | 0.00/2.20k [00:00<?, ?B/s]
```

```
model.safetensors: reconstructing file:   0%|          |  0.00B /  308MB
```

```
model.safetensors: downloading bytes:           |  0.00B
```

```
Loading weights:   0%|          | 0/190 [00:00<?, ?it/s]
```

```
generation_config.json:   0%|          | 0.00/147 [00:00<?, ?B/s]
```

```
c:\Users\ahmad\miniconda3\Lib\site-packages\transformers\generation\utils.py:1955:
UserWarning: Using the model-agnostic default `max_length` (=21) to control the generation
length. We recommend setting `max_new_tokens` to control the maximum length of the generation.
```

```
Je s'agissant de la technologie de linguage naturelle.
```

Part C — Encoder-Only Model

Encoder-only example: Sentiment Classification

Encoder-only models read the full input and create a representation of it.

They are very useful when the task is to **understand or classify text**.

Here we use a DistilBERT sentiment model.

```python
classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")

text = "This NLP lab is easy and interesting."

result = classifier(text)
print(result)
```

```
config.json:   0%|          | 0.00/629 [00:00<?, ?B/s]
```

```
model.safetensors: reconstructing file:   0%|          |  0.00B /  268MB
```

```
model.safetensors: downloading bytes:           |  0.00B
```

```
Loading weights:   0%|          | 0/104 [00:00<?, ?it/s]
```

```
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
```

```
vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]
```

```
[{'label': 'POSITIVE', 'score': 0.999778687953949}]
```

Tasks

Complete the following tasks.

Task 1 — Model Families

For each task below, choose the most suitable model family:

1. Sentiment classification
2. Machine translation
3. Open-ended text generation
4. Named Entity Recognition
5. Text summarization

```python
# Task 1: the most suitable model family for each task
import pandas as pd

answers = [
    ("1. Sentiment classification",
     "Encoder-only (BERT / DistilBERT)",
     "The whole sentence is read at once and turned into one representation that is "
     "then classified; no new text has to be generated."),

    ("2. Machine translation",
     "Encoder-decoder (T5 / BART / MarianMT)",
     "The encoder reads the source sentence and the decoder writes the target sentence,
"
     "so the input and the output are two different sequences."),

    ("3. Open-ended text generation",
     "Decoder-only (GPT family)",
     "The model only has to continue the text it has seen, predicting one token after "
     "another from the previous tokens."),

    ("4. Named Entity Recognition",
     "Encoder-only (BERT / DistilBERT)",
     "Each token is labelled using the context on both its left and its right, which is
"
     "exactly what a bidirectional encoder provides."),

    ("5. Text summarization",
     "Encoder-decoder (T5 / BART)",
     "The encoder reads the long document and the decoder produces a shorter text, so "
     "again the output is a new sequence conditioned on the input."),
]

df_answers = pd.DataFrame(answers, columns=["Task", "Model family", "Reason"])

for task, family, reason in answers:
    print(f"{task}")
    print(f"   Family : {family}")
    print(f"   Reason : {reason}\n")

df_answers[["Task", "Model family"]]
```

```
1. Sentiment classification
   Family : Encoder-only (BERT / DistilBERT)
   Reason : The whole sentence is read at once and turned into one representation that is then
classified; no new text has to be generated.

2. Machine translation
   Family : Encoder-decoder (T5 / BART / MarianMT)
   Reason : The encoder reads the source sentence and the decoder writes the target sentence,
so the input and the output are two different sequences.

3. Open-ended text generation
   Family : Decoder-only (GPT family)
   Reason : The model only has to continue the text it has seen, predicting one token after
another from the previous tokens.

4. Named Entity Recognition
   Family : Encoder-only (BERT / DistilBERT)
   Reason : Each token is labelled using the context on both its left and its right, which is
exactly what a bidirectional encoder provides.

5. Text summarization
   Family : Encoder-decoder (T5 / BART)
   Reason : The encoder reads the long document and the decoder produces a shorter text, so
again the output is a new sequence conditioned on the input.
```

	Task	Model family
0	1. Sentiment classification	Encoder-only (BERT / DistilBERT)
1	2. Machine translation	Encoder-decoder (T5 / BART / MarianMT)
2	3. Open-ended text generation	Decoder-only (GPT family)
3	4. Named Entity Recognition	Encoder-only (BERT / DistilBERT)
4	5. Text summarization	Encoder-decoder (T5 / BART)

Task 2 — Encoder-only Model

Use the sentiment classifier on these three sentences:

`The movie was excellent.`
`The lecture was difficult to understand.`
`I enjoyed learning about transformers.`

Print the predicted label and score for each sentence.

```
# Task 2: use the encoder-only sentiment classifier on three sentences
sentences = [
    "The movie was excellent.",
    "The lecture was difficult to understand.",
    "I enjoyed learning about transformers.",
]

results = classifier(sentences)

for sentence, result in zip(sentences, results):
    print(f"Sentence : {sentence}")
    print(f"Label    : {result['label']}")
    print(f"Score    : {result['score']:.4f}")
    print("-" * 60)
```

```
Sentence : The movie was excellent.
Label    : POSITIVE
Score    : 0.9999
------------------------------------------------------------
Sentence : The lecture was difficult to understand.
Label    : NEGATIVE
Score    : 0.9997
------------------------------------------------------------
Sentence : I enjoyed learning about transformers.
Label    : POSITIVE
Score    : 0.9997
------------------------------------------------------------
```

Task 3 — Encoder–Decoder Model

Use the FLAN-T5 model to perform the following:

- Summarize a short paragraph.

```python
# Task 3: use the encoder-decoder model (FLAN-T5) to summarize a short paragraph
paragraph = (
    "Natural language processing is a field of artificial intelligence that gives "
    "computers the ability to read, understand and produce human language. It combines "
    "ideas from linguistics, computer science and machine learning. Modern NLP systems "
    "are built on large neural networks called transformers, which are trained on very "
    "large amounts of text. These systems are used every day in machine translation, "
    "search engines, spam filters, chatbots and voice assistants."
)

prompt = "Summarize the following paragraph: " + paragraph

inputs = tokenizer(prompt, return_tensors="pt")
output = model.generate(**inputs, max_new_tokens=60)

print("Original paragraph:\n")
print(paragraph)
print("\n" + "=" * 70)
print("\nSummary produced by FLAN-T5:\n")
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

```
Original paragraph:

Natural language processing is a field of artificial intelligence that gives computers the
ability to read, understand and produce human language. It combines ideas from linguistics,
computer science and machine learning. Modern NLP systems are built on large neural networks
called transformers, which are trained on very large amounts of text. These systems are used
every day in machine translation, search engines, spam filters, chatbots and voice assistants.

======================================================================

Summary produced by FLAN-T5:

Learn about natural language processing. Learn about the use of natural language processing.
```

Task 5 — Short Reflection

Answer in 3–5 sentences:

Why is a decoder-only model suitable for a chatbot such as ChatGPT?

Answer

A decoder-only model is trained to predict the next token from all the tokens that came before it, and a conversation is exactly that: everything said so far is the context, and the reply is simply the continuation of that text. Because the training objective is only "predict the next word", the model can be trained on enormous amounts of ordinary text without anyone having to label it, which is what makes models of this size possible at all. Generating one token at a time also means the answer can be of any length and the model itself decides when to stop. Finally, since questions, instructions and answers are all just text, a single decoder-only model can handle translation, summarization, explanation and coding without a separate output head for each task.

Optional Challenge

Try generating text several times with the same prompt.

Remove:

`set_seed(42)`

and run the sampling cell multiple times.

Question

Why do you sometimes get different outputs even when the prompt is the same?

Hint: Sampling introduces randomness when choosing the next token.

```
# Optional Challenge: run the same prompt several times, with and without sampling
prompt = "Natural language processing is"

print("Greedy decoding (do_sample=False) - the output is always the same:\n")
for run in range(1, 4):
    out = generator(prompt, max_new_tokens=25, do_sample=False,
                    pad_token_id=generator.tokenizer.eos_token_id)
    print(f"Run {run}: {out[0]['generated_text']}\n")

print("=" * 75)
print("\nSampling (do_sample=True) - the output can be different every run:\n")
for run in range(1, 4):
    out = generator(prompt, max_new_tokens=25, do_sample=True, top_k=50,
                    pad_token_id=generator.tokenizer.eos_token_id)
    print(f"Run {run}: {out[0]['generated_text']}\n")
```

```
Greedy decoding (do_sample=False) - the output is always the same:

Run 1: Natural language processing is a common feature of the language processing process.

Run 2: Natural language processing is a common feature of the language processing process.

Run 3: Natural language processing is a common feature of the language processing process.

===========================================================================

Sampling (do_sample=True) - the output can be different every run:

Run 1: Natural language processing is one of the most important factors in your medical
professional's development, and is a critical part of your health.

Run 2: Natural language processing is a relatively new and complex language. Language
processing is the basis for most of the modern languages. The language is not only a

Run 3: Natural language processing is often one of the basic things about "learning" language.
It usually involves several components that are learned in a given language so
```

Answer

At every step the model does not produce a single word; it produces a probability distribution over the whole vocabulary. With `do_sample=False` the model always takes the single most likely token (greedy decoding), so the same prompt always gives the same output. With `do_sample=True` the next token is drawn **at random** from that distribution, so a slightly less likely word can be chosen, and from that point the two runs continue down completely different paths. `set_seed(42)` fixes the random number

generator, which makes the same sequence of random choices repeat, so removing it is what makes the output change from one run to the next.

Summary of the Tasks

Task 1 — Model families.

Task	Suitable family	Why
Sentiment classification	Encoder-only	The whole sentence is read at once and compressed into one representation that is classified; no new text is generated.
Machine translation	Encoder–decoder	The encoder reads the source sentence, the decoder writes a different sentence in the target language.
Open-ended text generation	Decoder-only	The model only continues the text it has already seen, one token at a time.
Named Entity Recognition	Encoder-only	Every token is labelled using the context on both sides of it, which is what a bidirectional encoder provides.
Text summarization	Encoder–decoder	A long input is read by the encoder and a shorter new sequence is produced by the decoder.

Task 2 — Encoder-only model. DistilBERT fine-tuned on SST-2 classified the three sentences with very high confidence:

Sentence	Label	Score
The movie was excellent.	POSITIVE	0.9999
The lecture was difficult to understand.	NEGATIVE	0.9997
I enjoyed learning about transformers.	POSITIVE	0.9997

Task 3 — Encoder–decoder model. FLAN-T5-small returned "*Learn about natural language processing. Learn about the use of natural language processing.*" The output is on topic but repetitive and loses most of the content of the paragraph. This is expected: *flan-t5-small* has only about 80 million parameters, which is enough to recognise the instruction but not enough to compress a paragraph well. The translation example earlier in the lab shows the same limitation — the French output is grammatically wrong. A larger checkpoint such as *flan-t5-base* or *flan-t5-large* produces noticeably better summaries with exactly the same code.

Task 5 — Why a decoder-only model suits a chatbot. A conversation is a single growing piece of text: everything said so far is the context and the reply is its continuation, which is precisely what a decoder-only model is trained to produce. Its training objective is only "predict the next token", so it can learn from enormous amounts of unlabelled text. Generating one token at a time lets the answer be of any length and lets the model decide when to stop. And because instructions, questions and answers are all just text, one model covers every task without a task-specific output layer.

Optional Challenge — why the output changes. The experiment makes the difference visible: with `do_sample=False` all three runs returned the identical sentence, while with

`do_sample=True` the three runs produced three completely different continuations. At each step the model outputs a probability distribution over the whole vocabulary. Greedy decoding always takes the highest-probability token, so the result is deterministic. Sampling draws the next token at random from that distribution, so one slightly less likely choice sends the rest of the sentence down a different path. `set_seed(42)` fixes the random number generator and makes the same sequence of random choices repeat, which is why removing it lets the output vary from one run to the next.